IIT JODHPUR

Feb'26 Minor Examination: EEL1570 Hardware Software Co-design (OPEN-BOOK)

Guidelines (Total time: 120 minutes, Maximum Marks: 25):

- Please read the question paper very carefully. **NO clarification is required in any question.** In case of any doubt, assume whatever you wish to and state that clearly in your answer.
- Usage of AI tools is NOT allowed.

---

1. You are tasked with designing an embedded vision system that continuously captures images from a CMOS camera sensor and displays them on an external LCD panel such that the displayed frames appear as real-time video. The camera captures images at a resolution of 1920 × 1080 pixels with 24-bit RGB color depth and a frame rate of 30 frames per second. Each raw frame must be compressed using a lossy compression algorithm achieving an average compression ratio of 8:1 before being stored in on-board DDR memory of 512 MB capacity.

   During playback, the system must read compressed frames from memory, decompress them in real time, remove the dark night-sky background using a pixel-wise segmentation algorithm, and apply a brightness and color transformation to simulate daylight conditions. The processed frames must then be displayed on the screen at 30 frames per second without frame drops. Based on these specifications:

   - Calculate the required raw data rate from the camera in MB/s.[1]
   - Estimate the minimum memory bandwidth needed to support simultaneous read (for display) and write (for capture) operations.[1.5]
   - Assuming the background removal algorithm requires 50 arithmetic operations per pixel and decompression requires 30 operations per pixel, compute the total number of operations per second required for real-time processing. [1.5]
   - Based on the computed bandwidth and computational requirements as above, evaluate whether a pure software (CPU-only), pure hardware (FPGA/ASIC-only), or hardware–software co-design approach would be most suitable. Justify your choice quantitatively in terms of processing throughput, latency, and memory bandwidth. [3]

2. Consider the design of a digital text-processing accelerator that receives an ASCII character stream from a communication interface at a sustained rate of 50 MB/s, where each character is 8 bits. The system performs three sequential operations on the incoming data stream. First, a case normalization block converts uppercase letters to lowercase and requires 2 clock cycles per character. Second, a keyword detection block compares each character against predefined keyword patterns and requires 6 clock cycles per character. Third, a word counter block increments a counter whenever a valid word boundary (space or newline) is detected, requiring 8 clock cycles per detected word. Assume that the average word length is 5 characters, and the system operates at a clock frequency of 200 MHz. Input buffering is limited, so the system must process data in real time without loss.

   Based on this specification, draw the Data-Flow Graph (DFG) of the system showing all computational nodes and data dependencies. Annotate each node with its execution latency and indicate whether it operates at character-level or word-level granularity. Determine the critical path of the DFG and calculate whether a single processing unit per stage can sustain the required 50 MB/s throughput at 200 MHz. If not, estimate the minimum parallelism required to achieve real-time operation. [2 + 3 + 1]

3. Consider the design of a digital audio signal processing system intended for a portable speech-enhancement device. The system processes a mono audio input sampled at 48 kHz, with each sample represented using 16-bit signed fixed-point format. The hardware platform runs at 24 MHz and has below resource constraints:

   - Maximum 4 hardware multipliers

- Maximum 3 hardware adders
- One absolute-value unit
- One squaring unit (can be implemented using a multiplier)
- No hardware division unit
- All operators are fully pipelined with a latency of 1 clock cycle

For every input sample, the system performs the following operations as shown below:

1. A **16-tap FIR filter** for noise suppression, described by

$$y[n] = \sum_{k=0}^{15} h[k] \cdot x[n-k]$$

2. A **dynamic range compression stage**, which computes

$$z[n] = \alpha \cdot y[n] + \beta \cdot |y[n]|$$

where α and β are constants.

3. An **energy estimator**, computed over a window of 64 samples as

$$E[n] = \sum_{k=0}^{63} z^2[n-k]$$

Assume real-time processing is required, meaning each input sample must be completely processed before the next sample arrives, from the system description, derive expressions for:
a) The total number of multiplications required per input sample, and
b) The total number of additions required per input sample.
Considering the given hardware resource limits (4 multipliers and 3 adders), determine whether the system can meet real-time processing requirements? If not, identify which operator type becomes the bottleneck. Suggest a scheduling or resource-sharing strategy that allows this design to satisfy the hardware constraints while maintaining real-time operation? [2+3+2]

4. Consider the design of an automotive driver-assistance controller for a mid-range vehicle, where a front-facing sensor module provides fused radar and camera data at a rate of 100 frames per second. For each frame, the system must perform the following tasks: (i) a 64-point digital filtering operation on each of 128 detected object signals for noise suppression, (ii) a feature extraction stage requiring 200 arithmetic operations per object, (iii) a rule-based decision algorithm that evaluates 50 logical conditions per frame to determine collision risk, and (iv) a CAN message packaging and transmission task to communicate braking or warning commands. The system operates with a main embedded processor running at 400 MHz, capable of executing one instruction per cycle, and an FPGA fabric that can implement up to 64 parallel multipliers and 64 adders. The total power budget for this module is limited to 5 W, and the braking decision must be generated within 5 ms of frame arrival to meet automotive safety timing constraints.

   You are required to perform a hardware–software partitioning analysis. Based on the computational workload, latency requirement, and available hardware resources, determine which tasks should be implemented in software on the processor and which should be accelerated in hardware on the FPGA. Estimate the total number of arithmetic operations per frame, compute the available clock cycles within the 5 ms deadline, and evaluate whether a software-only implementation can satisfy real-time constraints. Justify your final partitioning decision quantitatively in terms of execution time, parallelism, latency, and power considerations. [5]
